claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_cb1d52fa-fed\agent-adbc87b26cabc3c26.jsonl`
- SHA-256: `29e9be09621413a6a0a3376ee3054fe12cc11cffd90effad2e7631b275c56013`
- Source modified: `2026-06-14T08:26:54+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_cb1d52fa-fed`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T08:24:06.517Z)

THE PIVOT under study = "PERSONALIZATION-FIRST": instead of one universal model for all users, ship a BATTERIES-INCLUDED baseline that CONTINUALLY SELF-OPTIMIZES to a SINGLE user's own footage. As the user's corpus grows (10-15 videos), the on-device model + PER-USER CUE SELECTION (driven by the EXISTING LOVO / A-B experiment harness run on the user's OWN videos, with the user's CORRECTION LOOP as the per-user label source) becomes heavily tuned for THAT user ? deep analysis of THEIR rallies + shot types (serves, smashes, drives, drop shots). Per-user decisions like "the motion/person cue gives no F1 win for THIS user -> default it OFF for them." The model is optimized for the user's videos, NOT others (wins may hold anecdotally on other footage but weaker). Periodic "BATTERIES UPGRADES" improve the shipped baseline so it needs merely <5 videos to reach superior quality. Two user archetypes: (a) an economically-generous hobbyist who records sessions some days; (b) a professional with 100+ hours of their own video. CONTRAST = "generalization-first" (build one model that works on everyone's videos). The owner believes the harness + eval + A/B were DESIGNED to enable this roadmap.

DIMENSION: THE "BATTERIES UPGRADE" MECHANISM + improving the baseline. Analyze: how do you improve the shipped baseline so it needs <5 videos to reach superior quality, WITHOUT a central lake of user video (privacy)? Options: a Roora-labeled + owner-recorded seed corpus; pooled OPT-IN contributed data; FEDERATED learning (aggregate per-user model updates, not raw video); synthetic augmentation. How to MEASURE "<5 videos to superior quality" (a held-out-USER benchmark: train on k of a new user's videos, eval on the rest, vs the prior baseline). How to VERSION the baseline + PRESERVE each user's personalization across an upgrade (don't wipe their tuning). Return pros/cons + the upgrade-cadence design + the metric.

2. assistant (2026-06-14T08:24:11.997Z)

I'll analyze the "BATTERIES UPGRADE" mechanism for the personalization-first pivot. Let me first ground myself in the actual harness, eval, and correction-loop machinery that exists in this repo, since the design must build on real infrastructure.

3. user (2026-06-14T08:24:15.207Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-14T08:24:15.221Z)

1	# Next steps (live) ? owned-model + multisport
2
3	**Updated:** 2026-06-13 (**rally-detection quality track**: fusion + experiment-harness design docs merged,
4	owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior: 2026-06-10 (audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
5	ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-first delivery ? see
6	`docs/UI_ALPHA_EXECUTION_PLAN.md`; **PR-1 async jobs (#16) MERGED 2026-06-11, PR #87**;

**PR-2
7 upload/download MERGED 2026-06-11, PR #99** ? live-E2E-verified (3-part chunked upload
MD5 byte-identity +
8 compile?browser-download); next: PR-3 identity. Same day: the **8?9 quality sweep
#90?#98 landed** ? ruff/mypy/
9 coverage-floor CI lanes, SQLite conn-close fix, config unknown-key warning, hybrid-twin
collapse into
10 `trajectory_hybrid.py`, WSL tilde+abspath live-path fixes; suite 388 green; LOVO 0.626
re-verified exact).
11 **Authoritative roadmap:**
12 `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+ ADR-008 for topology). **Latest blow-by-
blow:** memory
13 `current-state.md` / `session-handoff.md`. This file = the prioritized "what to do
next," refreshed each session.
14
15 ## Where we are (one paragraph)
16 ADR-008 ratified ("repo split driven by productionization, not isolation"): training
lives in-repo (`training/`)
17 behind a CI-enforced import wall; the featurizer is single-sourced
(`backend/features/rally_seq.py`, train==serve);
18 the **Gen-0 harness ships** (`python -m training.gen0.harness --manifest
output/human_lovo_manifest.json --floor
19 eval_baselines/heuristic_lovo_n6.json --version <semver>`) and produced artifact v0.1.0
(LOVO **0.626**, floor passed,
20 sign test 5/6 wins p=0.109 ? honestly not significant, ship=False). The **n=6 owned
corpus is in the collector**
21 (262 rallies, Proprietary, commercial export = owned data only after the ShuttleSet
reclassification). The **Gemini
22 battery is done** (see table) ? the moat is quantified.
23
24 ## The quality table that now drives strategy (IoU 0.5 vs human golden labels)
25 | Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |
26 |---|---|---|
27 | Heuristic (velocity windowing) | 0.657 | 0.157 |
28 | **Gen-0 owned head** (LOVO, n=6) | 0.744 | 0.561 |
29 | Two-stage WASB+Gemini refine | 0.83 | ? |
30 | **Gemini wrapper** (`gemini-flash-latest`) | **0.93** | **0.66** |
31
32 **Reading:** clean footage is commoditized (serve it with Gemini for cents). **Multi-
court drops the commodity
33 wrapper to 0.66 ? that is the owned model's ship target** (its FPs are background-game
pickups + dead-time merges,
34 the exact contrast-metric confound). Gen-0 is 0.10 behind with only n=6 ? a corpus-
growth-sized gap, not an
35 architecture-sized one. Two-stage refine = the cost-efficient serving path for LONG
tapes (uploads candidate clips
36 only); wrapper 503-flakiness (observed all session) makes the provider-fallback registry
load-bearing.
37 Baselines tracked in `eval_baselines/` (heuristic_lovo_n6.json,
gemini_wrapper_2026-06-10.json).
38
39 ## Rally-detection quality track (NEW 2026-06-13 ? design merged, owner-gated, NOT
started)
40 **This is the track a desktop/local agent is slated to pick up.** Three design docs
landed (PR #112 =
41 fusion + Roora v8; this session = experiment harness + handoff). All implementation is
owner-gated.
42 - **Docs:** `docs/MULTI_SIGNAL_FUSION_PLAN.md` (shuttle + person + audio cues, one
fusion layer) ·
43 `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
44 `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md` (mental
model).
45 - **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a
rally" bug is
46 **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already

implements the
47   net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills service-
feed/held-shuttle
48   windows, but it is **only called from eval drivers** (`distill_local.py`,
`eval_partial_labels.py`,
49   `export_wasb_segments.py`, `calibrate_local.py`) and there is **no `min_crossings`
knob in
50   `IndexingConfig`** ? so production never applies it. **Wiring Gate A into the live
segmenter is the
51   single cheapest, highest-confidence quality win.**
52   - **Pick-up order (each = ONE PR off `master`, owner approval first):**
53   1. **Fusion P2 first-step ? wire Gate A into production:** add `min_crossings` to
`IndexingConfig`;
54   apply `rally_gate` in `wasb_hybrid`/`trajectory_hybrid` runtime (today it's eval-
only). Pure-logic,
55   testable in this container. *Fixes the observed bug.*
56   2. ? **DONE (2026-06-13) ? Fusion P1 ? extract the person cue:** lifted torchvision
detector + ByteTrack
57   + velocity out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py` (`PersonDetector`);
58   `yolo_hybrid` now orchestrates only + delegates to `self.person` (no behavior
change ? regression-safe,
59   proven by unchanged process_video tests). `tests/test_person_detector.py` added;
person_detector
60   type-clean. **The fusion segmenter (P2) reuses this producer over shuttle-seeded
windows.**
61   3. ? **DONE (2026-06-13) ? Experiment-harness P1 ? `backend/eval/experiment.py`:**
thin wrapper over
62   the EXISTING `calibration.py`, `metrics.py`, `classifier.py`, `training.py`,
`regression.gt_hash`, and
63   `database.query_candidate_segments` (no rebuilt scoring). Shipped `Variant` (+
pinned `CONTROL`),
64   `compare` (labeled A/B, LOVO-honest for learned variants), `compare_unlabeled`
(**SxS on new
65   unlabeled footage** ? agreed/A-only/B-only), `record_experiment` (leaderboard
JSON), and the DB
66   bridge; 19 tests incl. the no-regression invariant. Suite 421?440 green.
67   4. ? **DONE (2026-06-14) ? Fusion P2 ? the FusionSegmenter:**
`fusion_hybrid.py::FusionSegmenter`
68   (registry `fusion_hybrid`, **default-OFF**) seeds windows from the shuttle
substrate, persists
69   `net_crossings` (Gate A) + optional person features (`fusion_features.py`, pure-
tested) via two
70   identity hooks on the trajectory engine, with optional served Gate A/B
(`fusion.min_crossings` /
71   `min_players`, default 0) that gates the handoff while persisting the full
superset. Court mask =
72   optional `fusion.court_polygon`. Adversarially reviewed (31 agents, 19 confirmed;
NO-REGRESSION
73   verified); suite 441?467 green. Next: **P3** learned fusion (person features ?
harness `compare`
74   LOVO lift on the 6 golden videos; the eager-person-compute optimisation lands
here), then **P4**
75   audio cue via `backend/pipeline/audio/hit_detector.py`.
76   - **Discipline (from EXPERIMENT_HARNESS.md):** every new cue lands **flag-gated, default
OFF**; promote
77   to default **only on measured LOVO lift** through the `run_regression.py` gate (exit
0/1/3); pinned
78   control variant ? **rollback = config flip, never `git revert`**. The harness is ~80%
existing code.
79   - ? Dev container has no GPU/torch/cv2 ? Gate-A wiring, the person-cue extraction logic,
and
80   `experiment.py` are pure-logic/testable here; real-video confirmation is owner-side
(GPU/WSL).
81

Prioritized next steps

1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move; every lever below feeds on it.
   Priority order for new videos: **(a) multi-court badminton** (the target distribution AND where the ship target
   lives), **(b) ?3 matches per new sport ? table tennis first** (WASB transfers at F1 0.909; pickleball labels are
   corpus-gold but not trainable until a clean MIT/Apache ball detector exists), (c) capture variety per
   `VIDEO_RECORDING_GUIDELINES.md`; **adult sessions only for the training pool** (consent guardrail). Per video:
   label ? `curate import-local --normalize --license Proprietary --fps <true fps>` ? re-run the Gen-0 harness.
   The paired sign test needs n: at n=10 with 9 wins, p?0.011 ? significance is reachable this month.
2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480 (necessary), **multi-court
   reference = wrapper 0.66** (the number that makes the owned model worth serving), clean-footage ceiling = 0.93
   (not the target ? Gemini serves that tier). Suggested shape: "LOVO F1@tIoU0.5 ? 0.70 on multi-court folds with
   paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for **ActionFormer/ASFormer** (MIT, minutes on the
   2070S ? still lightweight, NOT the heavyweight Gen-2 stack) inside the existing harness; per-video threshold
   calibration (the LOVO-vs-oracle gap is visible per fold in the harness output).
4. **Multisport thread:** ingest Roora pickleball/TT CSVs (`import-local --normalize`); merged-prototype LOVO across
   badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-on-WASB serving stays gated
   by C3 (provenance multiplies per sport).
5. **PMF validation (cheaper than any engineering month):** C13 academy interviews (+ the two added questions:
   "multi-court footage nobody watches?" / "pay per-match to index it?"); camera pilot at the warmest 2-3 academies
   (consent at capture; demo served via the Gemini path or human-polish ? the reel pattern exists:
   `output/MahadevpuraSingles_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner ratified the
   physical-visit approach; reframed product-feedback-first per owner spec):**
   `docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md` (the recruiting flyer ? owner fills ?/contact placeholders and recruits) +
   `docs/FIELD_VISIT_PLAYBOOK.md` (the hired associate's visit playbook: coach + student script, demo-after-Q6
   rule, ground rules, Phase-2 record-and-show-back stretch goal) + `docs/FIELD_VISIT_ANSWER_SHEET.md` (per-visit
   capture form incl. demo-reaction + student blocks) + `docs/PMF_FIELD_SIGNALS.md` (G/A/R anchors + **pre-registered
   decision rules** ? PROCEED / PIVOT A-C / FALSIFIED thresholds fixed before the first visit).
6. **Housekeeping:** ~~rotate the Gemini API key~~ **DONE 2026-06-10** (owner-verified working; confirm old-key
   deletion in AI Studio); **rename repo off "badminton"** (owner action,
   in BACKLOG); collector=SoR confirmation; off-peak niceties: none pending ? the battery completed.

Pointers / artifacts

- Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Topology: ADR-008 (`docs/archives/decisions/DECISIONS.md`) ·
  Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` · Quality history:
  `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
- Harness: `training/gen0/harness.py` (one command ? train ? LOVO ? gate ? artifact bundle; weights gitignored,
  manifest reviewable). Featurizer: `backend/features/rally_seq.py` (FEATURE_SCHEMA v1).

```
120     - Corpus: collector DB (system-of-record, 6 matches/262 rallies) · eval manifest
`output/human_lovo_manifest.json`
121       (gitignored) · durable trajectories + Roora files: `C:\Users\avidu\Projects\Annotation
Setup\Collect\Trajectories\` ·
122       labels: `...\Collect\Golden Labelled\`.
123     - Gemini ops learnings: serial uploads (`indexing.ai_max_workers=1`) + chunk re-encode
124       (`indexing.ai_chunk_scale_width=1280`) + 6-attempt exponential generate retry ? a
cold/prepaid project burst-throttles
125       with a MISLEADING "credits depleted" 429; ramp gradually.
126     - ? No owned-model implementation beyond greenlit M0.x until owner greenlights the next
milestone. Committed next
127       PLATFORM slice = async job model (single-tenant).
128
```

5. user (2026-06-14T08:24:15.222Z)

```
1       # Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by
design)
2
3       **Status:** **P1 IMPLEMENTED** (`backend/eval/experiment.py` +
`tests/test_experiment.py`, shipped
4       2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest),
`compare_unlabeled()`
5       (SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.**
Phases below are
6       sequenced but each is a separate PR.
7
8       > **Companion docs:** the cues this experiments *on* are in
9       > [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); the scoring mechanics are
10      > [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels +
`baseline.json` come from
11      > [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md); the modular-cue architecture is
**ADR-007**
12      > ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the distilled
classifier is
13      > **ADR-004**. The 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).
14
15      ---
16
17      ## 1. Context ? why this, why now
18
19      PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues,
all **default
20      OFF**. The owner's directive for *how* we evolve quality from here:
21
22      > *"Evolve the codebase so we can keep finding new ways to improve quality and
experiment with them ?
23      > or add them as a signal ? without suffering any regressions if the idea doesn't work
out. Rolling
24      > back the code would be too dangerous."*
25
26      So **experimentation and deployment must be decoupled.** The contract:
27
28      1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production
behavior.
29      2. Test it **offline against golden labels** (most experiments never touch the served
path at all).
30      3. Promote a variant to default **only on measured lift**, through a CI eval gate.
31      4. **"Rollback" = flip a config flag, never `git revert`.** The proven path is a pinned
variant that is
32         always present.
33
34      **The pleasant surprise (Explore audit, 2026-06-13):** ~80% of this harness already
exists and is
```

```
35      simply not composed *as a system*. The missing piece is a thin **variant/experiment
layer + the
36      discipline**, not a platform. §4 is the exact code surface so the next session does not
re-discover it.
37
38      ---
39
40      ## 2. The thesis ? separate expensive DETECTION from cheap DECISION
41
42      The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per
frame" (shuttle
43      WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python
part is the
44      **decision**: "given those signals, where are the rallies" (windowing + gates + the
classifier).
45
46      > **Run detection once per video, persist all per-cue signals + per-candidate features,
then re-score
47      > any number of variants OFFLINE ? deterministically, with no GPU and zero production
risk.**
48
49      This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do
exactly this on
50      stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence
substrate.
51      ? Most experiments are pure re-scoring of stored data; they never enter the served
pipeline.
52
53      ---
54
55      ## 3. What already exists (the audit ? do NOT rebuild)
56
57      Exact public surface, with `file:line`. **This table is the anti-duplicate-work
artifact** ? start here.
58
59      ### 3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
60      `backend/eval/metrics.py`
61      - `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
62      - `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1
IoU matching.
63      - `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1,
mean IoU, boundary errors; `Score.as_dict()`.
64      - `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).
65
66      ### 3.2 A/B + cross-validation primitives (the comparison engine already exists)
67      `backend/eval/calibration.py`
68      - `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5)
-> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**
69      - `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113)
? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
70      - `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) ->
dict` (:72) ? within-video k-fold overfit guard.
71      - `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config,
float)` (:61).
72      - `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
73      - LOVO video dict shape: `{"name": str, "predict_fn": PredictFn, "gts": [Interval]}`.
74
75      ### 3.3 No-regression gate + baselines (the promotion gate already exists)
76      `backend/eval/regression.py`
77      - `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None,
tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ?
score stored preds vs GT, flag if P **or** R drops > tolerance.
78      - `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
79      - `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)`
(:80) · `load_baselines(path) -> dict` (:68).
```

80   - `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R,
`tolerance`, `has_baseline`; `as_dict()`.
81   - Default baseline file: `eval_baselines/regression_baseline.json` (:33).
82
83   `run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1
regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations
--stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-
missing-baseline --json`.
84
85   ### 3.4 Pinnable model artifact
86   `backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2):
`fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75),
`predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON)
(:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**
87
88   ### 3.5 Offline re-scoring substrate (persist once, re-score forever)
89   `backend/infrastructure/database.py`
90   - `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None,
confidence=None, stats=None, detection_params=None) -> Optional[int]` (:240) ?
`stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`,
`detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
91   - `query_candidate_segments(video_id, detector=None, only_unlabeled=False) ->
List[dict]` (:277) ? returns rows with deserialized `stats`.
92
93   ### 3.6 Label assignment + feature glue
94   `backend/eval/training.py`
95   - `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]`
(:50) ? y-labels by IoU match to golden (same matcher as the evaluator).
96   - `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5,
feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.
97
98   ### 3.7 Existing LOVO drivers to mirror (not duplicate)
99   - `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)`
(:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`;
`FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40);
already does LOVO over Gemini silver labels.
100   - `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5, metric="f1")` (:77);
`LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; grid + LOVO.
101   - `backend/eval/calibrate_wasb.py` ? `run(...)`, `load_golden_gts()`;
`backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden
reader.
102
103   ### 3.8 Registries + config knobs (variant selection)
104   - `backend/pipeline/segmenters/base.py` ? `SegmenterRegistry` (:35), singleton
`segmenter_registry` (:63); registered: `motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`,
`wasb_hybrid`.
105   - `backend/annotations/base.py` ? `AnnotationProviderRegistry`, singleton
`annotation_registry`; providers `file`, `auto`.
106   - `backend/config/models.py::IndexingConfig` ? `default_segmenter` (:65, default
`"yolo_hybrid"`), `detector_model` (:72), `detector_score_threshold` (:73),
`yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75), `min_segment_duration` (:68),
`dead_time_merge_gap` (:69), `motion_threshold` (:67). **No `min_crossings` knob yet** (see
fusion P2).
107
108   ### 3.9 Confirmed ABSENT (no duplication risk)
109   Explore found **no** existing experiment / variant / shadow / A-B *machinery* ? only a
stray "A/B test"
110   aspiration comment in `backend/pipeline/detectors/base.py` and an "experiment E1" note
in
111   `AUDIO_RALLY_DETECTION_PLAN.md`. The `regress()` + baseline path is the canonical
comparison mechanism.
112
113   ---
114
115   ## 4. The variant abstraction (the one thin thing to add)

A **`Variant` = a declarative recipe**, not a code branch:

```
Variant = (segmenter_name, IndexingConfig overrides, enabled cues + per-cue params, classifier model path/hash)
```

JSON-serializable; selected via the existing `segmenter_registry` + `IndexingConfig.default_segmenter`.
Control = a *pinned, frozen* variant (recipe + model hash) that is always registered and is the
default. Adding a new cue = a new Variant differing by one flag ? the control recipe is untouched.

---

5. Three experiment modes

5.1 Offline A/B (primary, label-driven, zero production risk)
`compare(videos, variant_A, variant_B)` ? a thin driver composing **existing** functions:
`query_candidate_segments()` (load persisted features) ? re-score each variant ? `calibration.improvement_report()`
+ `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden (`gt_loader.load_gt_intervals`).
Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B test, and it needs
almost no new scoring code** ? only the glue.

Honest reporting (default ON, additive): `compare(..., honest_stats=True)` also attaches a `"honest"`
block ? per-metric bootstrap **CIs**, a paired **?F1 CI + exact sign test** (C1), and **F1@k +
segment-count ratio** (C4) ? *alongside* the bare-mean fields (existing output unchanged; `honest_stats=False`
restores the legacy shape). A bare LOVO mean at n?6 is not promotable on its own; the lift is real when the
?F1 CI clears 0 **and** the sign test agrees. Wiring of the course corrections in
ANALYZERS_AND_RECONCILERS.md §13/C1+C4 (helpers: `backend/eval/eval_stats.py`,
`segmentation_metrics.py`); `record_experiment` carries the honest ?F1 into the leaderboard.

5.2 Shadow (for cues that touch detection, e.g. the person detector)
The new cue runs and **persists its signal** into the candidate `stats`/`detection_params`, but the
served decision stays control. Offline you then ask "what would variant B have scored?" via §5.1 ?
B never affects output until it wins. This is how a detection-touching change (not just a re-scoring
change) earns its way in without risk.

5.3 Promotion gate (no-regression, CI-enforced)
`regression.regress()` + `baseline.json` + `run_regression.py` exit codes. A variant becomes the new
default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1 blocks). Control is
pinned ? **rollback = flip `default_segmenter`/variant config, never a code revert.**

---

6. No-regression guarantees (the heart of the owner's ask)

- **New cues/signals default OFF** ? per-cue enable flag in `IndexingConfig`; merged

code can't change
162   behavior until explicitly enabled.
163   - **Control variant pinned** ? frozen recipe + classifier model hash, always registered
& default.
164   - **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no
silent default change.
165   - **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set
hash + timestamp,
166   persisted to a small JSON **experiment leaderboard** (generalizes `baseline.json`).
Answers "which
167   signals ever helped, on which video slices," and makes every result reproducible.
168   - **Decoupled events** ? "merge an experiment" and "change the default" are separate,
independently
169   revertible actions.
170
171   ---
172
173   ## 7. What to build later (gap list ? thin, additive, owner-gated)
174
175   1. ~~**`backend/eval/experiment.py`** ? a `Variant` dataclass + `compare(videos, a, b)`
composing the §3
176   functions (no new scoring math).~~ **DONE (2026-06-13).** Shipped `Variant` (+ pinned
`CONTROL`),
177   `compare` (labeled A/B with LOVO-honest learned-variant training, mirroring
`distill_local`),
178   `compare_unlabeled` (the **SxS-on-new-videos** mode ? agreed / A-only / B-only via
`match_intervals`,
179   no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), and
`load_video_candidates`
180   (the `query_candidate_segments` bridge). JSON variant recipes first as recommended; a
`VariantRegistry`
181   only if the count grows. 19 tests incl. the no-regression invariant.
182   2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`,
`two_sided_motion`,
183   `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`,
`direction_reversals`;
184   audio: hit times) so any future variant re-scores offline ? ties to
`MULTI_SIGNAL_FUSION_PLAN.md` §3.2/§4.
185   3. **Per-cue enable flags** in `IndexingConfig` (default `False`); `default_segmenter`
already exists.
186   Add the `min_crossings` knob (fusion P2) here too.
187   4. **CI promotion lane** using `run_regression.py` against the golden `baseline.json`;
write the
188   experiment-leaderboard JSON.
189
190   ---
191
192   ## 8. Phasing (owner-gated; ONE PR per slice, off `master`)
193
194   - **P0 ? this doc**, indexed in `docs/README.md`, cross-linked from
`MULTI_SIGNAL_FUSION_PLAN.md` and
195   `GOLDEN_SET_VENDORING.md`. *(this deliverable; docs-only, no code.)*
196   - **P1 ? `experiment.py` `compare()`** ? **DONE (2026-06-13):** thin wrapper over
`calibration` +
197   `metrics` + `query_candidate_segments`; pure-logic, fully tested in this dev container
(`compare`,
198   `compare_unlabeled`, `record_experiment`, the no-regression invariant).
`compare_unlabeled` adds the
199   SxS read on **new unlabeled footage** the owner asked for (run control + candidate on
a fresh video,
200   diff where they agree/diverge ? two reels via the existing `VideoCompiler` is the GPU-
side follow-on).
201   - **P2 ? per-cue enable flags + feature persistence** into candidate `stats` (default
OFF) ? couples
202   with fusion P1/P2.

```
203     - **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`;
experiment-leaderboard JSON.
204
205     ---
206
207     ## 9. Scope guardrail
208
209     **Single-tenant / local-first** (per `PLATFORM_ARCHITECTURE.md`): a JSON experiment
registry + the
210     existing harness ? explicitly **NOT** MLflow / W&B / a service. Keep it the size of
`baseline.json`.
211
212     ---
213
214     ## 10. Verification
215
216     - **Offline, runnable in this dev container (no GPU/torch needed for the glue):** unit
test that
217       `compare()` is deterministic on synthetic candidate rows; a **no-regression invariant
test** asserting
218       "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test
asserting
219       `run_regression.py` exit codes (0 ok / 1 regress / 3 no-baseline).
220     - **Machine-side (owner, GPU/WSL):** persist candidates once on a real clip, run
`compare(control,
221       fusion)` LOVO, confirm the reported lift matches a manual `metrics.score()` spot-
check.
222     - **Docs (P0):** links resolve; indexed in `docs/README.md`; cross-linked from
223       `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`.
224     </content>
225
```

6. assistant (2026-06-14T08:24:18.321Z)

Let me look at the actual harness, LOVO machinery, and correction-loop pieces that this upgrade design must build on.

7. user (2026-06-14T08:24:19.315Z)

```
1       """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3           python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4               --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6       Productionizes the rally_seq prototype per ADR-008:
7       - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8         honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9         the FINAL model on all videos.
10      - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11            <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12            <out>/<version>/manifest.json   (the reviewable contract: feature-schema version,
13                                             calibration, training lineage + GT hashes,
14                                             eval results, attestations, gate verdict)
15      - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16        (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17        (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18        undefined => no self-declared victory). The product side owns final admission
19        (backend/eval/regression.py); this harness never grades itself into serving.
20
21      Training may import backend.* (one-way wall); backend must never import training.*.
22      """
23      from __future__ import annotations
24
```

```
    import argparse
    import hashlib
    import json
    import os
    import subprocess
    import time
    from math import comb
    from typing import Dict, List, Optional

    import numpy as np

    from backend.eval.classifier import LogisticRegression
    from backend.eval.gt_loader import load_gt_intervals
    from backend.eval.regression import gt_hash
    from backend.eval.rally_seq_proto import (
        labels_for, run as lovo_run, _seg_f1,
    )
    from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    # Post-processing constants ? part of the model's identity, recorded in the manifest.
    CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
    THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]


    def _git_sha() -> str:
        try:
            return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
                                  text=True, timeout=10).stdout.strip() or "unknown"
        except Exception:
            return "unknown"


    def _sha256(path: str) -> str:
        h = hashlib.sha256()
        with open(path, "rb") as f:
            for chunk in iter(lambda: f.read(65536), b""):
                h.update(chunk)
        return h.hexdigest()


    def sign_test(learned: List[float], floor: List[float]):
        """Paired one-sided sign test: is learned > floor more often than chance?

        At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
        wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
        pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv]  # ties drop,
standard
        wins = sum(1 for lv, fv in pairs if lv > fv)
        n = len(pairs)
        p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
        return wins, n, p


    def train_final(specs: List[Dict]):
        """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).

        The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
        vids = []
        for s in specs:
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
```

```
            X, times = features(arr, float(s["fps"]))
            gts = load_gt_intervals(s["labels"], strict=False)
            vids.append({"name": s["name"], "X": X, "times": times,
                         "y": labels_for(times, gts), "gts": gts})
        Xall = np.vstack([v["X"] for v in vids])
        yall = np.concatenate([v["y"] for v in vids])
        clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
        clf.fit(Xall, yall, feature_names=FEAT_NAMES)
        thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
        return clf, float(thr)


    def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
        """Pre-flight ship-gate. ship=True requires EVERY block to clear ? and two blocks
        (undefined F1@tIoU target; WASB C3 provenance) are open project-wide, so today this
        can only ever say 'floor passed, not shippable yet'. That is by design: honesty
        over a green light."""
        reasons: List[str] = []
        floor_passed = None
        sign = None
        if floor:
            floor_mean = float(floor["mean_f1"])
            floor_passed = eval_res["mean_f1"] > floor_mean
            if not floor_passed:
                reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
            per = floor.get("per_video", {})
            paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
            if paired:
                wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
                sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
                if p >= 0.05:
                    reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
                                   f"? grow the corpus before claiming superiority")
        else:
            reasons.append("no floor baseline provided ? nothing to beat")

        reasons.append("ship-gate F1@tIoU target UNDEFINED (OWNED_MODEL_IMPLEMENTATION_PLAN
M0.4) ? "
                       "beating the floor is necessary, not sufficient")
        reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
        return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}


    def run(manifest_path: str, out_dir: str, version: str,
            floor_path: Optional[str] = None) -> dict:
        with open(manifest_path, encoding="utf-8") as f:
            specs = json.load(f)
        floor = None
        if floor_path:
            with open(floor_path, encoding="utf-8") as f:
                floor = json.load(f)

        # 1) Honest LOVO eval (the number that matters).
        eval_res = lovo_run(specs)

        # 2) Final model on all videos.
        clf, thr = train_final(specs)

        # 3) Gate (pre-flight).
        verdict = gate_verdict(eval_res, floor)
```

```
145
146         # 4) Artifact bundle.
147         bundle_dir = os.path.join(out_dir, version)
148         os.makedirs(bundle_dir, exist_ok=True)
149         weights_path = os.path.join(bundle_dir, "weights.json")
150         payload = clf.to_dict()
151         payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
152         payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
153         with open(weights_path, "w", encoding="utf-8") as f:
154             json.dump(payload, f, indent=2)
155
156         manifest = {
157             "artifact": {
158                 "name": "rally_tal_gen0", "version": version,
159                 "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
160                 "git_sha": _git_sha(),
161                 "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
162             },
163             "feature_schema": FEATURE_SCHEMA,
164             "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
165                             "note": "threshold tuned train-side on the full corpus; "
166                                     "honest generalization = the LOVO eval below"},
167             "training_lineage": {
168                 "corpus_manifest": manifest_path,
169                 "split_protocol": "leave-one-video-out by match",
170                 "label_source": "human (rally-annotator golden CSVs)",
171                 "videos": [{"name": s["name"], "labels": s["labels"],
172                             "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False)),
173                             "fps": s["fps"]} for s in specs],
174             },
175             "eval": {
176                 "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
177                 "per_video": eval_res["per_video"],
178                 "mean_f1": eval_res["mean_f1"],
179                 "floor": floor,
180             },
181             "attestation": {
182                 "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
183                 "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
184                 "no_paid_llm_training_labels": True,
185                 "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
186             },
187             "gate": verdict,
188         }
189         manifest_out = os.path.join(bundle_dir, "manifest.json")
190         with open(manifest_out, "w", encoding="utf-8") as f:
191             json.dump(manifest, f, indent=2)
192
193         print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
194         print(f"weights : {weights_path}")
195         print(f"manifest: {manifest_out}")
196         print(f"GATE    : ship={verdict['ship']}  floor_passed={verdict['floor_passed']}  "
197               f"sign_test={verdict['sign_test']}")
198         for b in verdict["blockers"]:
199             print(f"  - {b}")
200         return manifest
201
202
203     def main() -> None:
204         ap = argparse.ArgumentParser(description=__doc__)
205         ap.add_argument("--manifest", required=True,
206                         help="corpus manifest JSON:
```

```
[{name,trajectory,fps,frame_width,labels}, ...]")
207         ap.add_argument("--floor", default=None,
208                         help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
209         ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
210         ap.add_argument("--version", required=True, help="semver for the artifact bundle")
211         args = ap.parse_args()
212         run(args.manifest, args.out, args.version, floor_path=args.floor)
213
214
215     if __name__ == "__main__":
216         main()
217
```

8. user (2026-06-14T08:24:19.742Z)

```
1       """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3       This is the measurement layer for per-video / per-corpus calibration. It answers
4       two questions honestly:
5
6       1. **"Did precision improve, by how much?"** ? `improvement_report` scores a
7          baseline config vs a tuned config on the same ground truth and reports the
8          before/after metrics and the % delta.
9
10      2. **"Is that gain real or overfit?"** ? calibration must be measured on data the
11         tuning never saw, otherwise you're just memorising. Two protocols:
12         - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13           train folds, score on the held-out fold. The **generalization_gap**
14           (train ? test) is the overfit alarm.
15         - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16           pick the config on all *other* videos, score on the held-out video. Reports
17           per-video held-out scores + mean±std. This is the number to trust.
18
19      Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20      List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
21      harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
22      """
23      from __future__ import annotations
24
25      import statistics as _st
26      from typing import Callable, Dict, List, Sequence, Tuple, Any
27
28      from backend.eval.metrics import score, Score, Interval
29
30      Config = Any                              # opaque to this module (e.g. a thresholds
dict)
31      PredictFn = Callable[[Config], List[Interval]]
32
33
34      def pct_improvement(before: float, after: float) -> float:
35          """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
36          if before <= 0:
37              return float("inf") if after > 0 else 0.0
38          return (after - before) / before * 100.0
39
40
41      def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
42          """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43          return score(preds, gts, iou_threshold)
44
45
46      def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47          """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
```

```
        if n <= 0:
            return []
        k = max(2, min(k, n))                # need >=2 folds; cap at n
        folds = [list(range(i, n, k)) for i in range(k)]   # round-robin ? balanced,
deterministic
        splits = []
        for f in range(k):
            test = folds[f]
            train = [i for i in range(n) if i not in set(test)]
            if test and train:
                splits.append((train, test))
        return splits


    def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
                    metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
        """Pick the config whose predictions maximise `metric` against `gts`."""
        best_cfg, best_val = None, -1.0
        for cfg in configs:
            val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
            if val > best_val:
                best_cfg, best_val = cfg, val
        return best_cfg, best_val


    def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
                       k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
        """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).

        Per fold: choose the best config on the train rallies, score it on the held-out
        rallies. A large `generalization_gap` (train >> test) means the thresholds are
        overfitting that video's quirks.

        Default metric is **recall** on purpose: predictions are global (whole-video
        windows), so scoring them against a *fold subset* of GT would unfairly count the
        other rallies' windows as false positives ? precision/F1 are not well-defined
        per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
        is the honest within-video generalization signal. For precision/F1, use
        `leave_one_video_out` (each video scored whole) or `improvement_report` on a
        held-out video.
        """
        splits = kfold_indices(len(gts), k)
        if not splits:
            return {"error": "not enough rallies for cross-validation", "n": len(gts)}
        train_scores, test_scores, picked = [], [], []
        for train_idx, test_idx in splits:
            gts_tr = [gts[i] for i in train_idx]
            gts_te = [gts[i] for i in test_idx]
            cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
            te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
            train_scores.append(tr)
            test_scores.append(te)
            picked.append(cfg)
        mean_test = _st.mean(test_scores)
        mean_train = _st.mean(train_scores)
        return {
            "metric": metric,
            "folds": len(splits),
            "mean_test": mean_test,
            "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
            "mean_train": mean_train,
            "generalization_gap": mean_train - mean_test,   # >~0.1 = likely overfitting
            "picked_configs": picked,
```

```
        }


    def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
                            metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
Any]:
        """Leave-one-video-out CV ? the honest metric as the corpus grows.

        `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
        For each held-out video: pick the config that's best *pooled across the other
        videos*, then score it on the held-out one. The mean held-out score is the
        "precision on unseen videos" number; per-video spread shows stability.
        """
        if len(videos) < 2:
            return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
        per_video = []
        for i, held in enumerate(videos):
            others = [v for j, v in enumerate(videos) if j != i]

            def pooled_score(cfg: Config, others: List[dict] = others) -> float:
                vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
metric)
                        for v in others]
                return _st.mean(vals)

            best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
t: t[1])
            held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
iou_threshold), metric)
            per_video.append({"video": held.get("name", f"video_{i}"),
                              "held_out": held_score, "train_pooled": best_train, "config":
best_cfg})
        held_scores = [p["held_out"] for p in per_video]
        train_scores = [p["train_pooled"] for p in per_video]
        return {
            "metric": metric,
            "mean_held_out": _st.mean(held_scores),
            "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
            "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
            "per_video": per_video,
        }


    def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
                           gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
        """The "+X% precision" headline: baseline vs tuned config on the same GT.

        NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
        or read the cross-validated `mean_test` from `cross_validate` instead.
        """
        b = score(predict_fn(baseline_config), gts, iou_threshold)
        t = score(predict_fn(tuned_config), gts, iou_threshold)
        return {
            "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
            "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
            "precision_delta_pts": t.precision - b.precision,
            "precision_delta_pct": pct_improvement(b.precision, t.precision),
            "f1_delta_pct": pct_improvement(b.f1, t.f1),
        }

```

9. assistant (2026-06-14T08:24:23.563Z)

The harness, LOVO, and gate machinery are clear. Let me look at the correction loop (the per-user label source), the experiment leaderboard / variant layer, and how the owned-model plan frames the seed corpus and versioning.

10. user (2026-06-14T08:24:25.561Z)

```
Exit code 2
/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `"'
```

11. user (2026-06-14T08:24:26.602Z)

```
Found 7 files
backend\infrastructure\database.py
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
backend\eval\eval_stats.py
backend\tools\export.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\reporting\harvest.py
```

12. user (2026-06-14T08:24:30.527Z)

```
-rw-r--r-- 1 avidu 197609 28380 Jun 14 11:42 C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/experiment.py

C:/Users/avidu/Projects/badminton-highlight-indexer/eval_baselines/:
total 24
drwxr-xr-x 1 avidu 197609    0 Jun 14 12:14 .
drwxr-xr-x 1 avidu 197609    0 Jun 14 12:10 ..
-rw-r--r-- 1 avidu 197609 2506 Jun 14 12:14 experiment_leaderboard.json
-rw-r--r-- 1 avidu 197609 1977 Jun 10 17:43 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609  689 Jun 10 15:57 heuristic_lovo_n6.json
```

13. user (2026-06-14T08:24:30.870Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`    ? `pct_improvement`
15
16      The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17      (per-frame signals ? run once on the GPU/WSL box, persisted into
18      `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19      signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20      given persisted candidate features it produces `List[Interval]` deterministically,
21      with no GPU and zero production risk. So most experiments are pure offline
22      re-scoring of stored data and never touch the served pipeline.
23
24      Three modes (`EXPERIMENT_HARNESS.md` §5):
25        - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
26          aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27          held-out loop, but variant-parameterised.
```

```
        - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
          variants agree / diverge (A-only, B-only, agreed). The divergences are what a
          human spot-checks (and what two highlight reels would show side by side).
        - the promotion gate stays `run_regression.py` + `regression_baseline.json`
          (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

    Pure + offline + unit-tested. The only DB-touching helper is
    `load_video_candidates`, which reads persisted candidates via
    `Database.query_candidate_segments`.
    """
    from __future__ import annotations

    import json
    import logging
    import os
    from dataclasses import dataclass, field
    from datetime import datetime, timezone
    from hashlib import sha1
    from typing import Any, Callable, Dict, List, Optional, Sequence

    from backend.eval.calibration import pct_improvement
    from backend.eval.classifier import LogisticRegression
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval.metrics import Interval, match_intervals, score
    from backend.eval.regression import gt_hash
    from backend.eval.training import label_candidates
    from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
    from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
    from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

    logger = logging.getLogger(__name__)

    # Where a comparison run appends one reproducible record. Tracked location (NOT under
    # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
    # regression.DEFAULT_BASELINE_PATH.
    DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
    _METRICS = ("precision", "recall", "f1", "mean_iou")


    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
            for the leaderboard and for selecting the served path (the existing
            `segmenter_registry` + `IndexingConfig` do the actual selection in production).
            New cues are recorded here default-OFF.
          - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
```

```
            ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
            `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
            windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
          - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
            whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
            cue persists that feature).
          - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
            `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
            required for `compare_unlabeled` on a learned variant.
          - ``feature_names`` ? the persisted features the learned filter consumes. When set
            WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
            (honest) ? the recipe, not a pinned artifact.
          - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
            overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
        """

        name: str
        segmenter: str = "wasb_hybrid"
        config_overrides: Dict[str, Any] = field(default_factory=dict)
        cue_flags: Dict[str, bool] = field(default_factory=dict)
        min_crossings: int = 0
        min_players: int = 0
        classifier_model: Optional[str] = None
        feature_names: Optional[List[str]] = None
        threshold: float = 0.5
        merge_gap: float = 1.0
        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
        # failure where a fixed 0.5 zeroed out a small-candidate video.
        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
        calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first

        def is_learned(self) -> bool:
            """True if this variant applies a learned classifier (pinned model or
recipe)."""
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
```

```
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


```

```
    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                       # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
```

```
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
            train on the OTHER videos, predict the held-out one. The number to trust.
          - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
        """
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")

        per_video: List[Dict[str, Any]] = []
        predictions: Dict[str, List[Interval]] = {}

        recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
        pinned_model = (LogisticRegression.load(variant.classifier_model)
                        if variant.classifier_model is not None else None)
        all_model = None
        if recipe_lovo and not honest:
            all_model = _train(variant, videos, iou)

        for i, vc in enumerate(videos):
            cal: Optional[Callable[[float], float]] = None
            thr: Optional[float] = None
            if recipe_lovo and honest:
```

```
                others = [v for j, v in enumerate(videos) if j != i]
                if not others:
                    raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
                model: Optional[LogisticRegression] = _train(variant, others, iou)
                if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
                    assert model is not None                       # _train never returns
None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                       # honest=False ? one model over all
videos
                model = all_model
            else:                                   # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None               # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
            predictions[vc.name] = preds

        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }


    def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
        """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out

```

```
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                        eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                              "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
```

```
baseline.
        all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

        result: Dict[str, Any] = {
            "iou": iou,
            "label_set_hash": gt_hash(all_gts),
            "num_videos": len(videos),
            "variant_a": eval_a,
            "variant_b": eval_b,
            "delta": delta,
            "delta_pct": delta_pct,
            "per_video_delta": per_video_delta,
        }
        if honest_stats:
            result["honest"] = honest_summary(videos, eval_a, eval_b)
        return result


    # ------------------------------------------------------------- SxS on unlabeled video


    def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                          agree_iou: float = 0.5) -> Dict[str, Any]:
        """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

        With no labels there is no lift to measure ? instead this surfaces where the two
        variants **agree vs diverge**, which is exactly what a human reviews (and what two
        highlight reels would show side by side):

          - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
          - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
          - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
            the human / a later golden label adjudicates).

        Both variants must be predictable without labels: a learned variant therefore needs
a
        pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
        `metrics.match_intervals` ? no new matching logic.
        """
        preds_a = predict(variant_a, video)
        preds_b = predict(variant_b, video)
        mr = match_intervals(preds_a, preds_b, agree_iou)

        agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
                  for m in mr.matches]
        agree_ious = [m.iou for m in mr.matches]
        a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
        b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

        def _dur(ivs: List[Interval]) -> float:
            return sum(e - s for s, e in ivs)

        return {
            "video": video.name,
            "agree_iou": agree_iou,
            "variant_a": variant_a.name,
            "variant_b": variant_b.name,
            "num_a": len(preds_a),
            "num_b": len(preds_b),
            "num_agreed": len(agreed),
            "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
```

```
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
                              feature_names: Sequence[str] = ()) -> VideoCandidates:
        """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.

        ``feature_names`` selects which persisted features to lift out of each row's
``stats``
        JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
        function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
        but works against any `Database` (including an in-memory one in tests).
        """
        rows = db.query_candidate_segments(video_id, detector=detector)
        intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
        features: Dict[str, List[float]] = {}
```

```
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

14. assistant (2026-06-14T08:24:38.820Z)

I now have the full picture of the experiment harness, the variant layer (spec_hash, pinnable classifier, leaderboard), the honest-stats wiring (bootstrap CI + paired sign test), and the Gen-0 harness. Let me check the owned-model plan for how it frames the seed corpus, versioning, and the correction loop, plus the leaderboard format and how human labels feed back.

15. user (2026-06-14T08:24:41.325Z)

```
29:- **Exclude minors; per-user training data needs a separate explicit opt-in; split data by
match.**
50:  and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So:
**head now ? VLM later**,
65:  even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration):
`play_segments` today has
104:shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
```

16. user (2026-06-14T08:24:42.127Z)


```
1       [
2         {
3           "timestamp": "2026-06-14T06:23:51.507393+00:00",
4           "iou": 0.5,
5           "label_set_hash": "50d98ffbc370",
6           "num_videos": 6,
7           "variant_a": {
8             "name": "shuttle_only",
9             "spec_hash": "47c867c36f2f",
10            "aggregate": {
11              "precision": 0.31807603780927995,
12              "recall": 0.3101947096042718,
13              "f1": 0.3068611057861412,
14              "mean_iou": 0.5791831889148886
15            }
16          },
17          "variant_b": {
18            "name": "shuttle_person_fusion",
19            "spec_hash": "253f4709b4c4",
20            "aggregate": {
21              "precision": 0.3617228001510626,
22              "recall": 0.2708939086387243,
23              "f1": 0.2863398846047612,
24              "mean_iou": 0.5690111885714347
25            }
26          },
27          "delta": {
28            "precision": 0.04364676234178266,
29            "recall": -0.0393008009655475,
30            "f1": -0.02052122118138,
31            "mean_iou": -0.01017200034345389
32          },
33          "honest_delta_f1": {
```

```
            "mean_delta": -0.02052122118137999,
            "ci_low": -0.16462806145542527,
            "ci_high": 0.16224446866060574,
            "n": 6.0,
            "ci_excludes_zero": false,
            "sign_test": {
              "wins": 2.0,
              "losses": 4.0,
              "ties": 0.0,
              "n_effective": 6.0,
              "p_value": 0.6875
            }
          }
        },
        {
          "timestamp": "2026-06-14T06:24:56.487711+00:00",
          "iou": 0.5,
          "label_set_hash": "50d98ffbc370",
          "num_videos": 6,
          "variant_a": {
            "name": "shuttle_person",
            "spec_hash": "ee89f1e41604",
            "aggregate": {
              "precision": 0.3617228001510626,
              "recall": 0.2708939086387243,
              "f1": 0.2863398846047612,
              "mean_iou": 0.5690111885714347
            }
          },
          "variant_b": {
            "name": "shuttle_person_audio",
            "spec_hash": "d68c39eba87c",
            "aggregate": {
              "precision": 0.4530798177857001,
              "recall": 0.3408138120839734,
              "f1": 0.3655826552797436,
              "mean_iou": 0.7074802852140344
            }
          },
          "delta": {
            "precision": 0.0913570176346375,
            "recall": 0.06991990344524907,
            "f1": 0.07924277067498242,
            "mean_iou": 0.13846909664259965
          },
          "honest_delta_f1": {
            "mean_delta": 0.07924277067498244,
            "ci_low": -0.053508315003560535,
            "ci_high": 0.2029045239922802,
            "n": 6.0,
            "ci_excludes_zero": false,
            "sign_test": {
              "wins": 4.0,
              "losses": 2.0,
              "ties": 0.0,
              "n_effective": 6.0,
              "p_value": 0.6875
            }
          }
        }
      ]
```


17. user (2026-06-14T08:24:42.399Z)


```
      {
```

```
2         "name": "heuristic_velocity_windowing_LOVO",
3         "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
4         "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
5         "mean_f1": 0.480,
6         "std": 0.242,
7         "per_video": {
8           "adarsh_avi_singles": 0.751,
9           "kushagra_singles": 0.157,
10          "largetest_doubles": 0.737,
11          "gbaaddy": 0.309,
12          "testlarge_short": 0.267,
13          "mahadevpura_singles": 0.657
14        }
15      }
16
```

18. assistant (2026-06-14T08:24:46.948Z)

I have everything I need. The infrastructure is concrete: the leaderboard, spec_hash, pinnable per-variant model, LOVO machinery, the paired sign-test, and the per-match split. Let me check the schema fields the owned-model plan defines for consent/training_opt_in and source, since these are the federation/pooling hooks.

19. user (2026-06-14T08:24:47.792Z)

```
95        separate service. **None exists today.**
96
97      ---
98
99      ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) ·
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL
ActivityNet-JSON · VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
110     6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112     data + calibration, not method.** Keep growing per new sport.
113
114     ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) ·
*greenlight item*
115     - **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training
violation is
116       **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV
labels ? `output/local_rally_model.json`)
117       and the threshold distillation in `calibrate_local.py`. (NB:
`backend/eval/training.py::label_candidates` is a *pure,
118       source-agnostic* function ? there is no store by that name; the earlier
```

"training.label_candidates" pointer was wrong.)
119   Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the
path that regenerates them is one

20. assistant (2026-06-14T08:24:51.730Z)

I now have a complete, grounded picture. The schema already reserves
`source(human|pseudo|vendor)`, `consent/training_opt_in`, and `split(BY MATCH)` ? these are
exactly the federation/pooling hooks. The leaderboard, spec_hash, pinnable per-variant
classifier, LOVO, and paired sign-test are all real. I have enough to deliver the full analysis.

21. user (2026-06-14T08:26:37.793Z)

Structured output provided successfully

22. assistant (2026-06-14T08:26:54.565Z)

I've completed the analysis of the "batteries upgrade" mechanism dimension and returned it via
StructuredOutput.

Key grounding from the actual codebase that shaped the analysis:

- **The two artifacts an upgrade must touch are already cleanly separated in code:** the BATTERY
is the versioned bundle from `training/gen0/harness.py`
(`<out>/<version>/{weights.json,manifest.json}` with `weights_sha256`, `git_sha`,
`feature_schema_version`, per-video `gt_hashes`), and PERSONALIZATION is a `Variant` in
`backend/eval/experiment.py` (`spec_hash` + a per-user `classifier_model` JSON +
`min_crossings`/`min_players`/`cue_flags`/`tune_threshold`/`calibrate`). Upgrade-without-wipe is
therefore an artifact-versioning problem, gated on `feature_schema_version` stability.

- **The <5-videos metric already has its machinery:** a held-out-USER (held-out-MATCH, since
split-by-match is enforced) learning curve via `calibration.leave_one_video_out`, with
promotion-grade stats from `experiment.honest_summary` (paired ?F1 bootstrap CI + sign test) and
`harness.sign_test`.

- **The dominant blocker is statistical power, not design:** at n=6 nothing is significant ? the
Gen-0 harness itself refuses to ship (sign test 5/6, p=0.109), and the two recorded fusion
experiments in `eval_baselines/experiment_leaderboard.json` are a 2/6 loss (p=0.6875) and a non-
significant 4/6 (?F1 CI [-0.054, +0.203] crosses 0).

- **Honest re-framing the doc should make:** the correction loop feeds personalization ~90% and
the shared battery only ~10% (via the noisy federated-gradient path); the real upgrade engine is
the seed + pooled-opt-in + synthetic corpus, and the held-out-user benchmark it optimizes is ?
ironically ? a generalization metric.

The four privacy-preserving sources, the minor-vs-major upgrade taxonomy (the planned M0.4
ActionFormer swap is a MAJOR one that invalidates every per-user 5-feature LogReg), the
federated-privacy hardening gaps (no DP, no cohort floor, consent flag reserved-but-unenforced),
the versioning/preservation design, and the cadence-gated-on-evidence proposal are all captured
in the structured fields with the relevant `file:line` anchors.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\training\gen0\harness.py`,
`C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\experiment.py`,
`C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibration.py`,
`C:\Users\avidu\Projects\badminton-highlight-
indexer\eval_baselines\experiment_leaderboard.json`, `C:\Users\avidu\Projects\badminton-
highlight-indexer\eval_baselines\heuristic_lovo_n6.json`, `C:\Users\avidu\Projects\badminton-
highlight-indexer\docs\EXPERIMENT_HARNESS.md`, `C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md`, `C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\NEXT_STEPS.md`.